



## LABORATORY 2

# ROS 2 data logging and ROV motion

Write a subscriber that records the BlueROV2 trajectory, then a publisher that commands a simple mission.

| Course       | INRO - Instrumentation, Marine Robotics and Power Systems |
|--------------|-----------------------------------------------------------|
| Professor    | Daniel Mihai Toma                                         |
| Prerequisite | Laboratory 1 and a working Dev Container                  |
| Main result  | Python code, CSV data and trajectory plots                |

### Mission

Record time, x, y, z and depth while your own Python publisher makes the ROV descend, move forward and return towards its initial depth.

This laboratory uses open-loop effort commands. Exact speed, position and depth control belong to Laboratory 3.

# 1. Learning outcomes

After this laboratory you should be able to:

- create a ROS 2 subscriber and explain when its callback runs;
- read position from `nav_msgs/Odometry` and depth from `std_msgs/Float64`;
- write time, x, y, z and depth to a CSV file;
- create a ROS 2 publisher and send `[surge, heave, yaw]` commands;
- plot and interpret the trajectory recorded during your mission.

## Laboratory sequence

| Part         | Student action                              | Result                       |
|--------------|---------------------------------------------|------------------------------|
| A. Observe   | Inspect one odometry and one depth message. | Fields and units identified. |
| B. Subscribe | Run and study a two-topic CSV logger.       | Trajectory CSV created.      |
| C. Publish   | Run and study a short motion sequence.      | ROV moves in the pool.       |
| D. Program   | Write personal logger and mission files.    | Student Python code.         |
| E. Report    | Plot x, y, z and the 3D path.               | Figures and interpretation.  |

Two ideas only  
Subscriber: data arrives and ROS calls your callback.  
Publisher: your program creates a message and sends it to a topic.

## Safety limits

- Run only one motion program at a time.
- Keep each effort between -0.40 and +0.40.
- Keep each motion at or below 5 seconds.
- Use these commands only with the simulation.

## 2. Start the simulator and inspect the topics

After pulling the new repository version, rebuild the Dev Container once so the plotting library is installed. Then launch the same simulator used in Laboratory 1:

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
python3 src/inro_rov_demo/scripts/prepare_model.py
colcon build --packages-select inro_rov_demo --symlink-install
source install/setup.bash
ros2 launch inro_rov_demo demo.launch.py
```

Keep this terminal running. Use `gui:=false` if your computer uses the headless profile.

### 2.1 Read one message

Open a second terminal, source the workspace, and run:

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
source install/setup.bash
ros2 topic info /inro/odometry
ros2 topic echo /inro/odometry --once
ros2 topic echo /inro/depth --once
```

| Value   | Where to find it            | Convention                                |
|---------|-----------------------------|-------------------------------------------|
| x, y, z | odometry.pose.pose.position | Gazebo world coordinates; z points upward |
| depth   | /inro/depth message data    | Metres below z = 0; positive downward     |
| time    | logger clock                | Seconds since the logger started          |

Before coding

Write down the message type for each topic. Which part of the odometry message contains x, y and z?

### 3. Subscriber and callback

A subscriber tells ROS which message type and topic to listen to, and which function should receive each new message.

```
self.create_subscription(
    Odometry, '/inro/odometry', self.odometry_callback, 10
)
```

When a new odometry message arrives, ROS calls `odometry_callback`. The callback reads the position:

```
def odometry_callback(self, msg):
    position = msg.pose.pose.position
    x = position.x
    y = position.y
    z = position.z
```

The complete example uses a second subscriber to remember the latest depth:

```
self.create_subscription(
    Float64, '/inro/depth', self.depth_callback, 10
)

def depth_callback(self, msg):
    self.latest_depth = msg.data
```

#### Callback rule

Keep callbacks short. Read the message, store or write the values you need, then return so ROS can process the next message.

Open `lab02/trajectory_logger.py`. Find both subscription calls, both callbacks, and the line that extracts `msg.pose.pose.position`.

## 4. Save the trajectory to CSV

The logger opens one file and writes a header before ROS begins processing messages:

```
self.csv_file = OUTPUT_FILE.open(
    'w', newline='', encoding='utf-8'
)
self.writer = csv.writer(self.csv_file)
self.writer.writerow(['time_s', 'x_m', 'y_m', 'z_m', 'depth_m'])
```

Each odometry callback adds one row:

```
self.writer.writerow([
    elapsed, position.x, position.y, position.z, depth
])
self.csv_file.flush()
```

Run the example in the second terminal and keep it active:

```
python3 src/inro_rov_demo/lab02/trajectory_logger.py
```

It prints one update per second but records every odometry message. Stop it with Ctrl+C after the motion experiment. The file is saved as:

```
ros2_ws/src/inro_rov_demo/lab02/rov_trajectory.csv
```

| time_s | x_m  | y_m  | z_m   | depth_m |
|--------|------|------|-------|---------|
| 0.000  | 0.00 | 0.00 | -2.00 | 2.00    |
| ...    | ...  | ...  | ...   | ...     |

Your first subscriber program

Create `my_trajectory_logger.py`. Use the example as a guide, but type the two subscription calls and callbacks yourself. Submit your own file with the report.

## 5. Publisher and motion command

The ROV command topic expects a `Float64MultiArray` with exactly three values:

Command order  
[surge, heave, yaw]  
Positive surge: forward | Positive heave: up | Positive yaw: turn left

A publisher sends the message:

```
publisher = node.create_publisher(
    Float64MultiArray, '/inro/command', 10
)
message = Float64MultiArray(data=[surge, heave, yaw])
publisher.publish(message)
```

The example publishes repeatedly for a fixed duration. It sends zero effort between motions and again before closing.

### Initial mission

| Motion       | surge | heave | yaw  | duration |
|--------------|-------|-------|------|----------|
| Descend      | 0.00  | -0.25 | 0.00 | 2.5 s    |
| Move forward | +0.30 | 0.00  | 0.00 | 3.0 s    |
| Ascend       | 0.00  | +0.25 | 0.00 | 2.5 s    |

python3 src/inro\_rov\_demo/lab02/motion\_sequence.py

Effort is not speed  
A command of 0.30 requests normalized thrust effort. It does not request 0.30 m/s or a particular distance.

## 6. Write and record your own mission

Create `my_motion_sequence.py`. Reuse the safe publishing function, but write your own MISSION list.

```
MISSION = [
    ('descend',      0.00, -0.25, 0.00, 2.5),
    ('move forward', 0.30,  0.00, 0.00, 3.0),
    ('ascend',       0.00, +0.25, 0.00, 2.5),
]
```

### Required sequence

- 1 Restart the simulator so every final run begins from the same pose.
- 2 Start `my_trajectory_logger.py` and check that live values appear.
- 3 Run `my_motion_sequence.py`.
- 4 Wait two seconds after the mission, then stop the logger with Ctrl+C.
- 5 Open the CSV and check that time, x, y, z and depth contain numerical values.

Complete this prediction table before running the final trial:

| Motion  | Expected x | Expected y | Expected z/depth | Observed |
|---------|------------|------------|------------------|----------|
| Descend |            |            |                  |          |
| Forward |            |            |                  |          |
| Ascend  |            |            |                  |          |

Optional 3D extension  
Add a short yaw command, then another forward command. The changed heading should make both x and y vary.

## 7. Plot, interpret and submit

Create the supplied report plots:

```
python3 src/inro_rov_demo/lab02/plot_trajectory.py
```

The script saves `rov_trajectory.png` beside the CSV. You may also open the CSV in Excel, LibreOffice Calc or another plotting tool.

### Your report

- your `my_trajectory_logger.py` and `my_motion_sequence.py`;
- the final CSV file;
- a plot of `x`, `y` and `z` versus time;
- a plot of depth versus time;
- one 3D trajectory plot;
- the completed prediction table and a short interpretation.

### Answer these questions

- Which callback writes each CSV row, and what event makes it run?
- Why do `z` and depth have opposite signs?
- Why does the ROV continue moving briefly after a zero command?
- Why can this open-loop mission not guarantee an exact final position?

### Troubleshooting

| Problem                                                                                                                                                | Check                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| No position values                                                                                                                                     | Start <code>demo.launch.py</code> and source <code>install/setup.bash</code> in the terminal. |
| No CSV appears                                                                                                                                         | Stop the logger with <code>Ctrl+C</code> and check the <code>lab02</code> source folder.      |
| No plot or matplotlib error                                                                                                                            | Pull the latest repository and rebuild the Dev Container once.                                |
| ROV does not move                                                                                                                                      | Run only one motion program and check <code>/inro/command</code> has one subscriber.          |
| ROV hits the floor                                                                                                                                     | Reduce the downward effort or duration; restart before the final trial.                       |
| Next<br>Laboratory 3 replaces timed open-loop effort with feedback control for requested speed and target depth, then studies the CTD density profile. |                                                                                               |

Simulation only: never connect these teaching scripts to the real ROV.